

Chapitre 4 : Conteneurs

TD / TP

Table des matières

1	Travaux Dirigés	2
1.1	Listes	2
1.1.1	Opérations	2
1.1.2	Découpage	2
1.1.3	Boucles	2
1.1.4	Opérations bis	2
1.2	Ensembles.....	3
1.2.1	Propriétés	3
1.2.2	Algorithme	3
1.2.3	Fonction	3
1.3	Dictionnaires	3
1.3.1	Manipulation.....	3
1.3.2	Compréhension	3
1.3.3	Boucles	4
1.3.4	Propriétés	4
1.3.5	Fusion	4
1.4	Conteneurs.....	4
1.4.1	Conversions.....	4
1.4.2	Liste en compréhension	4
2	Travaux Pratiques	4
2.1	retirer_indices.py	5
2.2	retirer_elements.py	6
2.3	retirer_doublons.py	7
2.4	multiensemble.py	8
2.5	evaluer.py	10
2.6	sous_ensembles.py	11
2.7	Remise	11
3	Travaux Pratiques – Suite	12

3.1	liste.py.....	12
3.2	triangle_pascal.py.....	13
3.3	arrangements.py.....	14

1 Travaux Dirigés

Vous devez réaliser les exercices suivants sur papier.

1.1 Listes

1.1.1 Opérations

Donnez la valeur de `ma_liste` après l'exécution des opérations suivantes :

```
ma_liste = [1, 2, 3, 4, 5]
ma_liste.append(6)
ma_liste.insert(2, 99)
ma_liste.pop(3)
ma_liste.remove(2)
ma_liste[1] = 7
```

1.1.2 Découpage

Soit la liste `nombre` = [10, 20, 30, 40, 50, 60, 70]. Quelle est la valeur des expressions suivantes ?

```
nombre[2:5]
```

```
nombre[::2]
```

```
nombre[-3:]
```

1.1.3 Boucles

Écrivez le code Python qui : (i) crée une liste de nombres de 1 à 10, (ii) supprime tous les nombres pairs de la liste, (iii) affiche la liste finale.

1.1.4 Opérations bis

Soit la liste suivante :

```
liste = [1, 2, 3]
```

En partant de cette même valeur, que contient `liste` après chacune des opérations suivantes ?

```
liste.append((4, 5, 6))
```

```
liste.extend((4, 5, 6))
```

1.2 Ensembles

1.2.1 Propriétés

Que se passe-t-il quand vous ajoutez à un ensemble un élément déjà présent dans l'ensemble ?

Pourquoi les ensembles ne peuvent-ils pas contenir de listes ?

1.2.2 Algorithme

Soient deux ensembles `e1` et `e2`. Ecrivez le code Python qui affiche les éléments présents dans `e1` mais pas dans `e2`, sans utiliser les méthodes ensemblistes de la classe `set`.

1.2.3 Fonction

Ecrivez la fonction suivante, sans utiliser les méthodes ensemblistes de la classe `set`.

```
def union_excl(e1: set, e2: set) -> set:
    """ Retourne un nouvel ensemble contenant les éléments présents dans e1
        et e2 mais pas les deux.

        e1 et e2 ne sont pas modifiés par la fonction.
    """
```

1.3 Dictionnaires

1.3.1 Manipulation

Soit le dictionnaire suivant :

```
etudiant = {"nom": "Alice", "age": 20, "notes": [12, 15, 18]}
```

Donnez les expressions ou instructions permettant de :

- accéder à la valeur associée à la clé « age »
- ajouter une nouvelle clé « ville » avec la valeur « Paris »
- ajouter la note 20 à la liste de note
- supprimer la clé « age » (et bien sûr la valeur associée)

1.3.2 Compréhension

Soit le dictionnaire suivant :

```
d = {"a": 1, "b": 2, "c": 2}
```

Que retournent les expressions suivantes ?

```
[x for x in d]
```

```
[x for x in d.keys()]
```

```
[x for x in d.values()]
```

```
[x for x in d.items()]
```

1.3.3 Boucles

Soit le dictionnaire suivant, associant des noms de fruits à leur prix :

```
fruits = {"pomme": 2.50, "poire": 3.70, "orange": 2.75, "peche": 4.25}
```

Ecrivez le code Python qui affiche pour chaque *fruit* son *prix* sous la forme : « Le prix de *fruit* est *prix* Euro. ». Donnez-en une seconde version.

1.3.4 Propriétés

Peut-on utiliser des listes comme clé dans un dictionnaire ? Peut-on utiliser des tuples ?

Peut-on utiliser des listes comme valeur dans un dictionnaire ? Peut-on utiliser des tuples ?

1.3.5 Fusion

Ecrivez la fonction suivante, sans utiliser la méthode `update()` de la classe dictionnaire :

```
def fusion(d1: dict, d2: dict) -> dict:
    """ Retourne un nouveau dictionnaire consistant en la fusion de d1 et d2.

    Il contient toutes les associations (clé, valeur) de d1 et de d2. Si
    une clé est présente dans d1 et d2, la valeur de d2 est retenue. d1
    et d2 ne sont pas modifiés par la fonction.
    """
```

1.4 Conteneurs

1.4.1 Conversions

Convertissez la liste `[1, 2, 2, 3, 4, 4, 5]` en un ensemble. Quel est le résultat, quelle instruction Python fait cette conversion ?

Convertissez l'ensemble `{1, 2, 3}` en une liste. Quel est le résultat, quelle instruction Python fait cette conversion ?

Convertissez la liste de tuples `[("a", 1), ("b", 2), ("c", 3)]` en un dictionnaire. Quel est le résultat, quelle instruction Python fait cette conversion ?

1.4.2 Liste en compréhension

Si elle est correcte, quelle est la valeur de l'expression suivante ?

```
[(i,j) for i in range(2) for j in range(3) if i != j]
```

2 Travaux Pratiques

Il s'agit de définir en Python les fonctions décrites dans les paragraphes suivants.

Le titre du paragraphe est le nom du script à remettre. Le texte qui suit spécifie la fonction à définir, en donnant son prototype, son docstring et des exemples d'exécution sous l'interpréteur Python.

Vous ne devez utiliser que des constructions et fonctions que nous avons étudiées en cours. Certains exercices peuvent spécifier des contraintes de réalisation supplémentaires.

Le testeur fourni ne teste que quelques cas représentatifs. Le testeur qui établira votre note testera des cas supplémentaires et vérifiera le respect des contraintes de réalisation.

Vos fonctions doivent tester la validité des *valeurs* de leurs paramètres, mais pas celle de leur *types* : le testeur passera toujours des arguments conformes en type aux paramètres.

Voici quelques éléments nouveaux par rapport aux précédents TP :

Vos fonctions peuvent s'appuyer si nécessaire sur des fonctions utilitaires *personnelles* que vous définissez dans le même script.

Comme indiqué dans leur spécifications, certaines fonctions ne modifient pas un ou plusieurs de leurs paramètres : soyez attentifs à cette contrainte.

L'annotation de type d'un conteneur spécifie entre crochets le type de ses éléments, s'il est fixé, ou any, s'il est quelconque. Pour un dictionnaire, on spécifie le type de ses clés et celui de ses valeurs, séparés par une virgule. Ainsi : `list[any]` est une liste d'éléments de type quelconque, `set[int]` un ensemble d'éléments de type `int` et `dict[any, int]` un dictionnaire dont les clés sont de type quelconque et les valeurs de type `int`.

2.1 retirer_indices.py

```
def retirer_indices(liste: list[any], indices: set[int]) -> int:
    """ Retire d'une liste des éléments d'indices donnés.

    Si un indice est hors limites, la fonction ne le traite pas et ne
    retourne pas d'erreur.

    Paramètres :

        liste : liste de laquelle retirer les éléments.

        indices : ensemble des indices d'éléments à retirer.

    Retourne :

        Le nombre d'éléments de la liste effectivement retirés.
    """
```

Exemples :

```
>>> liste = ['a', 'b', 'c']
>>> retirer_indices(liste, {0, 2})
2
>>> liste
['b']
>>> liste = ['a', 'b', 'c']
>>> retirer_indices(liste, {3, 0})
```

```

1
>>> liste
['b', 'c']
>>> liste = ['a', 'b', 'c']
>>> retirer_indices(liste, {0, 1, 2})
3
>>> liste
[]
>>> liste = ['a', 'b', 'c']
>>> retirer_indices(liste, set())
0
>>> liste
['a', 'b', 'c']
>>>

```

2.2 retirer_elements.py

```

def retirer_elements(liste: list[any], elements: set[any]) \
    -> dict[any, int]:
    """ Retire d'une liste un ensemble d'éléments donnés.

    La fonction retire toutes les occurrences d'un élément donné. Si
    un élément donné n'appartient pas à la liste, la fonction ne le
    traite pas et ne retourne pas d'erreur.

    Paramètres :

        liste : liste de laquelle retirer les éléments.

        elements : ensemble des éléments à retirer.

    Retourne :

        Un dictionnaire donnant, pour chaque élément à retirer (clé),
        le nombre d'occurrences retirées (valeur).
    """

```

Exemples :

```

>>> liste = ['a', 'b', 'a', 'c']
>>> retirer_elements(liste, {'a', 'd'})
{'a': 2, 'd': 0}
>>> liste
['b', 'c']
>>> liste = ['a', 'b', 'a', 'c']
>>> retirer_elements(liste, {'b'})
{'b': 1}
>>> liste
['a', 'a', 'c']

```

```
>>> liste = ['a', 'b', 'a', 'c']
>>> retirer_elements(liste, set())
{}
>>> liste
liste = ['a', 'b', 'a', 'c']
>>>
```

2.3 retirer_doublons.py

```
def retirer_doublons(liste: list[any], en_place: bool = True) \
    -> list[any]:

    """ Retire d'une liste les doublons d'éléments.

    La fonction retire toutes les occurrences de chaque élément sauf
    la première. La liste résultante contient des éléments uniques,
    dans l'ordre de leur première occurrence dans la liste initiale.

    La fonction soit effectue les modifications en place, soit génère
    une nouvelle liste en laissant la liste initiale non modifiée.

    Paramètres :

        liste : liste de laquelle retirer les doublons.

        en_place : True si l'opération est effectuée en place et False
        sinon.

    Retourne :

        La liste résultante. Si en_place est True, elle est le même
        objet que la liste passée en argument.

    """
```

Exemples :

```
>>> liste = [3, 2, 2, 1, 1, 3, 2]
>>> resultat = retirer_doublons(liste)
>>> resultat
[3, 2, 1]
>>> liste
[3, 2, 1]
>>> resultat is liste
True
>>> liste = [3, 2, 2, 1, 1, 3, 2]
>>> resultat = retirer_doublons(liste, False)
>>> resultat
[3, 2, 1]
>>> liste
[3, 2, 2, 1, 1, 3, 2]
```

```
>>> resultat is liste
False
>>>
```

2.4 multiensemble.py

Un *multiensemble* est une généralisation d'un ensemble dans laquelle les éléments peuvent apparaître plusieurs fois. Par exemple, {a, b, a, c, b} est un multiensemble contenant deux fois l'élément a, deux fois l'élément b et une fois l'élément c. On dit que la multiplicité de a dans l'ensemble est 2, celle de b, 2 également et celle de c, 1. Les opérations ensemblistes classiques – test d'inclusion, union, intersection, différence – se généralisent aisément aux multiensembles.

Comment représenter cette structure mathématique en Python ? Les ensembles ne le permettent évidemment pas. On pourrait penser utiliser une liste, qui peut contenir plusieurs fois un même élément : le multiensemble ci-dessus serait par exemple représenté par ['a', 'b', 'a', 'c', 'b']. Cependant une liste ne permet pas, de façon efficace, de tester la présence d'un élément ou d'établir sa multiplicité. (Par ailleurs elle est ordonnée, alors qu'un multiensemble ne l'est pas.) La structure qui répond parfaitement au besoin est un dictionnaire : ses clés sont les éléments du multiensemble et leur valeurs la multiplicité des éléments. Ainsi, le multiensemble ci-dessus peut être représenté par le dictionnaire {'a': 2, 'b': 2, 'c': 1}.

Dans cet exercice, il s'agit de définir, dans le fichier `multiensemble.py`, les fonctions de manipulation de multiensembles suivantes :

```
def ajouter(ensemble: dict[any, int], element: any) -> int:
    """ Ajoute un élément à un multiensemble.

    Paramètres :

        ensemble : ensemble dans lequel ajouter.

        element : élément à ajouter.

    Retourne :

        La multiplicité de l'élément dans l'ensemble après ajout.
    """
```

Exemples :

```
>>> e = {'a': 2, 'b': 2, 'c': 1}
>>> ajouter(e, 'd')
1
>>> e
{'a': 2, 'b': 2, 'c': 1, 'd': 1}
>>> ajouter(e, 'b')
3
>>> e
{'a': 2, 'b': 3, 'c': 1, 'd': 1}
```

```
>>>
```

```
def retirer(ensemble: dict[any, int], element: any) -> int:
    """ Retire un élément d'un multiensemble.

    Paramètres :

        ensemble : ensemble duquel retirer.

        element : élément à retirer.

    Retourne :

        La multiplicité de l'élément dans l'ensemble après retrait, ou
        -1 si l'élément était absent avant retrait.

    """
```

Exemples :

```
>>> e = {'a': 2, 'b': 2, 'c': 1}
>>> retirer(e, 'b')
1
>>> e
{'a': 2, 'b': 1, 'c': 1}
>>> retirer(e, 'c')
0
>>> e
{'a': 2, 'b': 1}
>>> retirer(e, 'd')
-1
>>>
```

```
def union(ensemble1: dict[any, int], ensemble2: dict[any, int]) \
    -> dict[any, int]:
    """ Retourne l'union de deux multiensembles.

    Paramètres :

        ensemble1 : premier opérande, non modifié par la fonction.

        ensemble2 : second opérande, non modifié par la fonction.

    Retourne :

        L'union des deux ensembles.

    """
```

Exemples :

```
>>> e1 = {'a': 2, 'b': 1}
```

```
>>> e2 = {'b': 1, 'c': 2}
>>> union(e1, e2)
{'a': 2, 'b': 2, 'c': 2}
>>> e1
{'a': 2, 'b': 1}
>>> e2
{'b': 1, 'c': 2}
>>>
```

2.5 evaluer.py

```
def evaluer(expression: list[any]) -> float:
```

```
    """ Evalue une expression arithmétique représentée par une liste.
```

```
    Les expressions que la fonction traite sont de la forme suivante :
```

```
    <expr> ::= "[" <oprd> "," <optr> "," <oprd> "]"
```

```
    <oprd> ::= <nbre> | <expr>
```

```
    <optr> ::= '+' | '-' | '/' | '*'
```

```
    <nbre> ::= nombre entier ou réel
```

```
    Paramètres :
```

```
        expression : expression à évaluer, non modifiée par la
                     fonction.
```

```
    Retourne :
```

```
        L'évaluation de l'expression, ou None si elle est mal formée ou
        conduit à une erreur de calcul.
```

```
    """
```

Exemples :

```
>>> evaluer([2.7, '+', 1])
3.7
>>> evaluer([[1, '+', 2], '*', 3])
9
>>> evaluer([[1, '+', 2], '*', [3, '-', 4.0]])
-3.0
>>> evaluer([1, 'x', 2])
>>> evaluer([1, '+', 'hi!'])
>>>
```

Aide : pour correctement traiter les opérandes, la fonction doit tester le type des éléments de la liste-expression. Cela peut se faire avec les expressions booléennes suivantes :

```
type(element) is int
type(element) is float
type(element) is list
```

2.6 sous_ensembles.py

```
def sous_ensembles(ensemble: set[any]) -> list[set[any]]:
    """ Retourne la liste des sous-ensembles d'un ensemble, classée
        par ordre de tailles croissantes.

    Paramètres :

        ensemble : ensemble à traiter, non modifié par la fonction.

    Retourne :

        La liste des sous-ensembles de l'ensemble, classée par ordre de
        tailles croissantes.
    """
```

Exemples :

```
>>> sous_ensembles({1})
[set(), {1}]
>>> sous_ensembles({1, 2})
[set(), {1}, {2}, {1, 2}]
>>> sous_ensembles({1, 2, 3})
[set(), {1}, {2}, {3}, {1, 2}, {1, 3}, {2, 3}, {1, 2, 3}]
>>>
```

Contrainte : la fonction ne fait appel à aucune fonction de tri. Son algorithme insère les sous-ensembles directement à leur position correcte dans la liste.

Remarques :

L'ordre des éléments dans un ensemble et l'ordre des sous-ensembles de même taille dans la liste sont indifférents. Une autre réponse correcte de `sous_ensembles({1, 2, 3})` est par exemple :

```
[set(), {3}, {1}, {2}, {2, 1}, {3, 2}, {1, 3}, {3, 2, 1}]
```

La fonction s'écrit de façon simple en s'appuyant sur la propriété suivante. On note E un ensemble, e un élément n'appartenant pas à E et $P(E)$ l'ensemble des sous-ensembles E . Alors :

$$P(\emptyset) = \{\emptyset\}$$

$$P(E \cup \{e\}) = P(E) \cup \{p \cup \{e\}, p \in P(E)\}$$

Dit de façon informelle : pour former l'ensemble des sous-ensembles de $P(E \cup \{e\})$, on prend ceux de E , et on leur ajoute les sous-ensembles formés en ajoutant l'élément e à chacun d'eux.

Mais vous pouvez bien sûr utiliser tout autre méthode pour calculer cet ensemble.

2.7 Remise

Remettez svp vos fichiers .py dans l'espace Moodle **Remise TP4**.

Vous devez les remettre à *plat*, c'est-à-dire sans répertoires ou sous-répertoires intermédiaires et sans les regrouper en archive. Il vous suffit de faire un simple glisser-déposer des différents fichiers depuis votre répertoire de développement dans l'espace Moodle.

Vous pouvez remettre vos fichiers autant de fois que vous le voulez jusqu'à la date limite. Toute nouvelle version doit *écraser* la précédente : les fichiers nommés par exemple `script(1).py` seront ignorés lors de la correction.

3 Travaux Pratiques – Suite

Ces nouveaux exercices sont à réaliser dans les mêmes conditions que précédemment, et remis dans l'espace Moodle **Remise TP4 - Suite**. Ils comptent autant que les TP précédents en termes d'évaluation.

3.1 liste.py

Comme indiqué en cours, un dictionnaire peut être vu comme une généralisation d'un tableau C ou d'une liste Python dans laquelle l'accès aux éléments se fait grâce à des *clés de type quelconque* et non plus seulement des *indices de type entier*.

Dans cet exercice, on met en pratique cette idée en utilisant un dictionnaire pour représenter une liste : les valeurs du dictionnaire, de type quelconque, sont les éléments de la liste, leur clés, de type entier, sont leur indices dans la liste. Par exemple la liste `['a', 'b', 'c']` est représentée par le dictionnaire `{0: 'a', 1: 'b', 2: 'c'}`.

Il s'agit de réimplémenter dans le fichier `liste.py` les principales fonctions de manipulation de liste suivantes, avec cette nouvelle représentation. Leur docstrings et leurs exemples d'utilisation ne sont pas redonnés car elles ont le même comportement que leur original de la classe `list`. Il n'y a que deux exceptions à cela : (a) les fonctions ne gèrent pas les indices négatifs et les considèrent comme erronés, (b) elles ne génèrent pas d'exception en cas d'erreur, mais retournent une valeur particulière, indiquée dans la description.

Remarque : pour bien implémenter ces fonctions, rappelez-vous que les clés d'un dictionnaire ne sont pas ordonnées : une boucle `for` sur un dictionnaire ne va pas nécessairement parcourir les clés dans l'ordre 0, 1, 2, etc.

```
def append(liste: dict[int, any], value: any) -> None:
def count(liste: dict[int, any], value: any) -> int:
def index(liste: dict[int, any], value: any, \
          start: int = 0, stop: int = 9223372036854775807) -> int:
```

La fonction retourne -1 si la valeur n'est pas trouvée.

```
def insert(liste: dict[int, any], index: int, value: any) -> None:
def pop(liste: dict[int, any], index: int) -> any:
```

La fonction retourne None si l'élément à retirer n'existe pas.

```
def remove(liste: dict[int, any], value: any) -> None:
```

La fonction retourne sans erreur si l'élément à retirer n'existe pas.

```
def reverse(liste: dict[int, any]) -> None:
```

3.2 triangle_pascal.py

Préambule : Le triangle de Pascal donne les coefficients binomiaux, i.e. les coefficients du développement de $(a + b)^n$. Voici par exemple le triangle de Pascal d'ordre 2 :

```
1
1    1
1    2    1
```

et celui d'ordre 3 :

```
1
1    1
1    2    1
1    3    3    1
```

On remarque que chaque ligne commence et termine par 1, et qu'entre ces deux valeurs, on a la relation : $c_{i,j} = c_{i-1,j-1} + c_{i-1,j}$ où i est le numéro de ligne et j le numéro de colonne d'un élément du triangle.

```
def triangle_pascal(ordre: int) -> list[list[int]]:
```

```
    """ Retourne les coefficients du triangle de Pascal d'ordre n.
```

```
    Paramètres :
```

```
        n : ordre du triangle à générer.
```

```
    Retourne :
```

```
        La liste des listes des coefficients, chaque liste représentant
        une ligne du triangle, ou None en cas d'erreur.
```

```
    """
```

Exemples :

```
>>> triangle_pascal(2)
[[1], [1, 1], [1, 2, 1]]
>>> triangle_pascal(3)
[[1], [1, 1], [1, 2, 1], [1, 3, 3, 1]]
>>>
```

3.3 arrangements.py

```
def arrangements(ensemble: set[any], p: int) -> list[list[any]]:
    """ Retourne la liste des arrangements à p éléments d'un ensemble.

    Paramètres :

        ensemble : ensemble d'éléments à arranger.

        p : taille des arrangements à générer.

    Retourne :

        La liste des arrangements de l'ensemble, ou None en cas d'erreur.
    """
```

Exemples :

```
>>> arrangements({1, 2, 3}, 1)
[[1], [2], [3]]
>>> arrangements({1, 2, 3}, 2)
[[1, 2], [1, 3], [2, 1], [2, 3], [3, 1], [3, 2]]
>>> arrangements({1, 2, 3}, 3)
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
>>>
```

Remarque : l'ordre des arrangements dans la liste est indifférent.